# Software Implementation and Testing Document

# For

# Group 1

Version 3.0

**Authors**:
Jaliah Meade
Nikki Zahedi
Isabella Maldur
Annika Needham

## 1. Programming Languages (5 points)

*List the programming languages use in your project, where you use them (what components of your project) and your reason for choosing them (whatever that may be).*

**Python handled the backend side of the application, taking care of routing, request processing, session management, and authentication, while also interacting with the database. SQL was used to set up and manage the database itself, organizing and storing data for things like employees, shifts, availability, and swap requests. On the front end, HTML was used to structure the web pages, including forms and layouts such as the login page, and CSS was used to style those pages and improve the overall visual design.**

## 2. Platforms, APIs, Databases, and other technologies used (5 points)

*List all the platforms, APIs, Databases, and any other technologies you use in your project and where you use them (in what components of your project).*

**Flask was used to build the web application and manage navigation between the different pages and components. SQLite served as the database system, handling the storage and organization of all the application's data.**

## 3. Execution-based Functional Testing (10 points)

*Describe how/if you performed functional testing for your project (i.e., tested for the **functional requirements** listed in your RD).*

**Functional testing was performed by checking that overlapping shift requests were handled correctly with existing schedules and that shifts could not be incorrectly double-booked. The shift swap feature was tested to ensure employees could submit swap requests properly, including cases where no specific employee was selected, allowing it to function as an open request. The shift swap page was also verified to confirm that the CSS rendered correctly and that the layout displayed as intended. On the admin side, testing confirmed that the dashboard features worked properly, including viewing and editing employee schedules and availability, and that the added UI elements functioned correctly and displayed the appropriate data.**

## 4. Execution-based Non-Functional Testing (10 points)

*Describe how/if you performed non-functional testing for your project (i.e., tested for the **non-functional requirements** listed in your RD).*

**Non-functional testing was performed by checking that the application loaded pages such as the login, dashboard, shift swap, and admin views efficiently and without noticeable delays. The system was also evaluated for responsiveness to ensure that user interactions, such as submitting forms or navigating between pages, behaved smoothly. Input validation was tested to confirm that invalid or missing data did not cause errors or crashes and was handled properly. In addition, the system was tested with multiple users interacting with features like shift swaps and availability updates to ensure stable performance and consistent behavior under simultaneous use.**

## 5. Non-Execution-based Testing (10 points)

*Describe how/if you performed non-execution-based testing (such as code reviews/inspections/walkthroughs).*

**Non-execution-based testing was done through code reviews and walkthroughs of key features like authentication, shift swaps, and admin functions, where the code was checked for logic errors, consistency, and proper integration between components.**